AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

23CSE203

DATA STRUCTURES AND ALGORITHM

Submitted by	
Name	B.SAI HASHINI
Roll.no	AV.SC.U4CSE25101
Section	CSE-B

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Credit	5

Index

S.no	Title	Page.no	Remark
1.	**A)** A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. **B)** Bank calculates the growth of an investment using compound intrest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n(power), where P=principal growth factor, n=no.of years. **C)** Write a recursive progrsm to print the first n terms of fibanocci series **D)** An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list. **E)** In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.	4-13	
2.	**A)** write a python program to perform linear search on the given input **B)** Write a python program to perform binary search on the given	14-18	

	input. Verify by taking sorted array and unsorted array as a list.		
3.	A) write a python program to implement bubble sort, where the elements are taken from user. B) write a python program to implement selection sort, where the elements are taken from user. C) write a python program to implement insertion sort, where the elements are taken from user.	19-	

Week-1

A) Aim: A rocket launch system displays a countdown before launch. Instead of using loop, implement the

countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch.

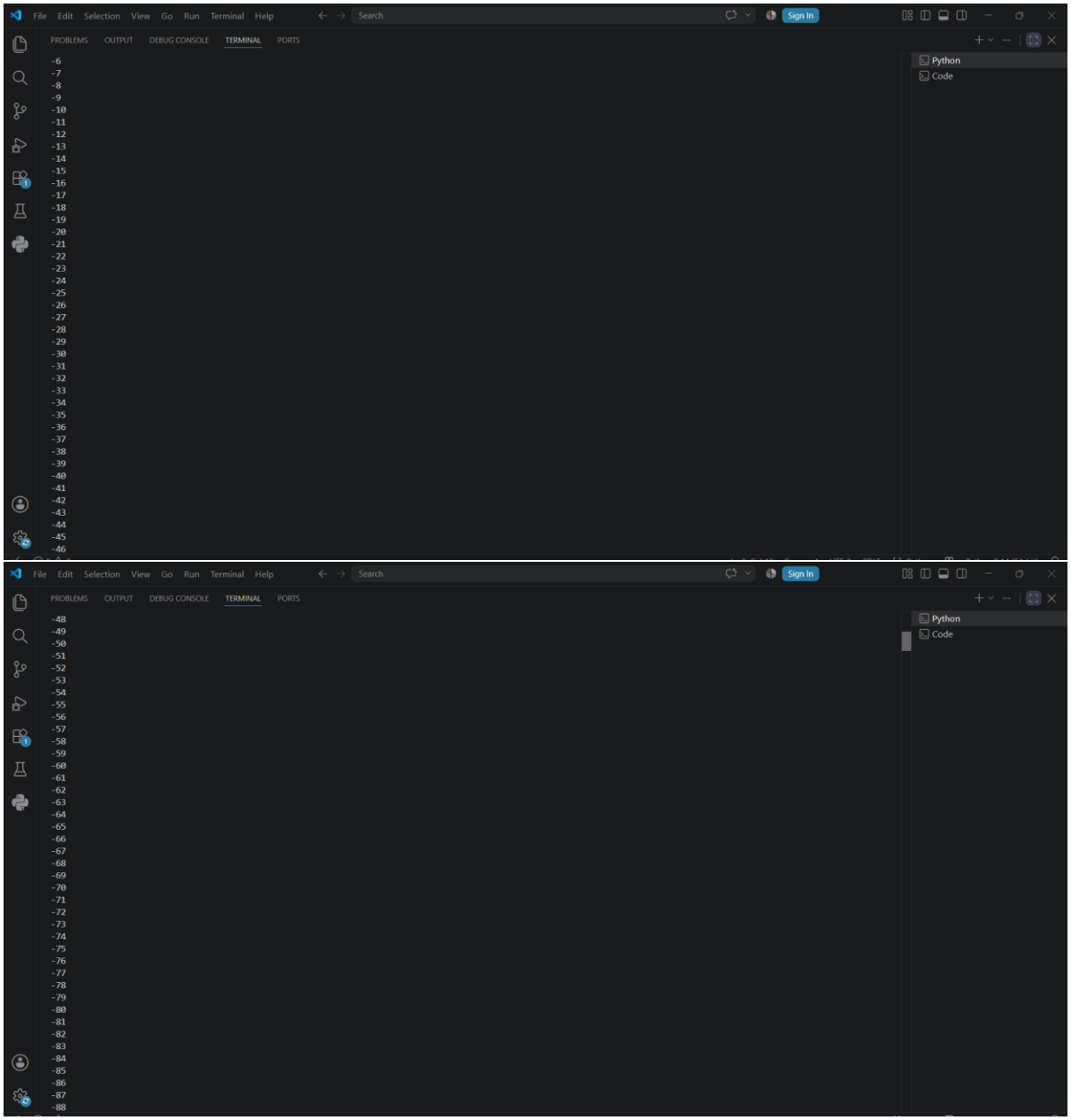
Program:

```
#av.sc.u4cse25101
def countdown(n):
    if n == 0:
        print("launch")
    else:
        print(n)
        countdown(n-1)

countdown(5)
```

Output:

```
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
ktop/textbooks/DSA Lab/week 1/LAB1.A.py"
5
4
3
2
1
launch
```



Explanation:

- It takes a number as input from the user and stores it in num.
- It creates a function named launch().
- It starts the countdown from 10 by assigning i = 10.
- It prints the value of i as long as num <= i.
- It decreases the value of i by 1 after each iteration.

- When the loop ends, it prints "Rocket launched" and executes the function using launch().
- For negative input, the function keeps decreasing instead of reaching 0, causing infinite recursion and eventually a `Recursion Error`.

Error table:

s.no	Error obtained	Rectification
1.	Infinite loop	Iteration at correct place
2.	Calling function at wrong place	Placed with correct syntax

B) Aim: Bank calculates the growth of an investment using compound intrest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n(power), where P=principal growth factor, n=no.of years.

Program:

C: > Users > SAI HASHINI > OneDrive > Desktop > textbooks > DSA Lab > week 1 > lab1.B.py > ..

```python
#av.sc.u4cse25101
def power(p, n):
    if n == 0:
        return 1
    elif n < 0:
        return 1 / power(p, -n)
    else:
        return p * power(p, n - 1)

principal = 1000
rate = 0.05
years = -3

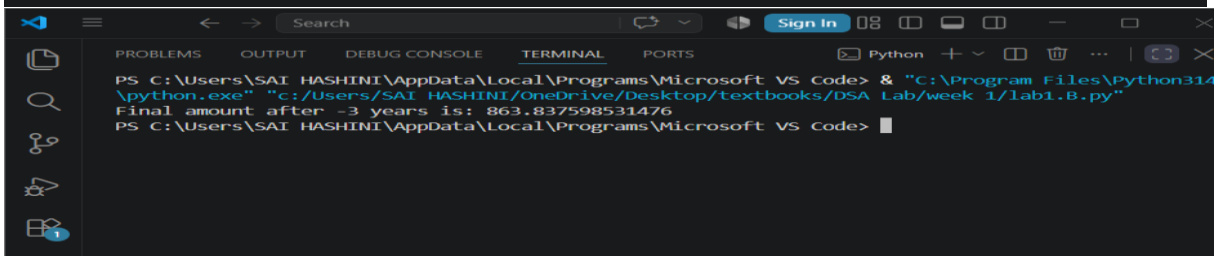
amount = principal * power(1 + rate, years)

print("Final amount after", years, "years is:", amount)

```

Output:

```
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
Final amount after 3 years is: 1157.6250000000002
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
```



Explanation:

- The program takes the principal growth factor p and number of years n as input.
- It defines a recursive function power(p, n).
- If n is 0, the function returns 1 (base case).
- Otherwise, it returns p * power(p, n-1).
- The recursive calls continue until n becomes 0.
- The final value of p^n is displayed as the output.

- For a negative exponent, calculate the reciprocal instead of continuing recursion downward indefinitely.
- Error table:

s.no	Error obtained	Rectification
1.	Return 0	Return 1
2.	print	return

C) Aim: Write a recursive program to print the first n terms of fibanocci series

Program:

```
#av.sc.u4cse25101
def fibonacci(n):
    if n < 0:
        return "Number of terms cannot be negative"
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

Output:

```
Enter number of terms: 5
0 1 1 2 3
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
```

```
Enter number of terms: -3
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>
```

Explanation:

- For entering the number as input from user we defined as n
- We declared variable for formula.
- If input number is less than one then it return “1”
- Or it will goes under fibanocci iteration.
- If x is not present in the list, recursion reaches the end and returns `False`.

Error:

s.no	Error obtained	Rectification
1.	Forgot to declare n<=1	rectified
2.	Formula mistake	rectified

D) Aim: An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list.

Program:

```python
#av.sc.u4cse25101
def search(a, x, i=0):
    if i == len(a):
        return False
    if a[i] == x:
        return True
    return search(a, x, i + 1)


ids = [101, 205, 310, 415, 520]

print(search(ids, 310))
```

Output:

```
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1> pyth
True
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1> 310
310
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
```

```
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314
\python.exe" "c:/Users/SAI HASHINI/OneDrive/Desktop/textbooks/DSA Lab/week 1/lab1.c.py"
False
```

Explanation:

- The program creates a function to search for an employee ID in a list.
- It checks one employee ID at a time, starting from the first ID.
- If the entered employee ID matches the current ID, it returns True.
- If it does not match, the function moves to the next ID using recursion.
- If all employee IDs are checked and no match is found, it returns False.

- Finally, the program displays "Employee ID found." if the ID exists; otherwise, it displays "Employee ID not found."
- Error:

s.no	Error obtained	Rectification
1.	Emp lst	Emp_list
2.	print	Return

E) Aim: In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.

Program:

```
#av.sc.u4cse25101
def factorial(n):
    if n < 0:
        return"not defined"
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
num = int(input("Enter a number: "))
result = factorial(num)
print(f"The factorial of {num} is {result}")

```

Output:

```
Enter a number: 5
The factorial of 5 is 120
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
```

```
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code> & "C:\Progr
\python.exe" "c:/Users/SAI HASHINI/OneDrive/Desktop/textbooks/DSA Lab/week 1/
Enter a number: -8
The factorial of -8 is not defined
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>
```

Explanation:

- The program defines a recursive function fact() to calculate the factorial of a number.
- It takes a number as input from the user.
- If the number is 1, the function returns 1 (base case).
- Otherwise, it multiplies the number by the factorial of the previous number (n-1).
- The recursive calls continue until the number becomes 1.
- Finally, the program prints the factorial of the given number.
- Negative numbers do not have a factorial, so the program should handle them separately instead of recursing forever.

Error table:

s.no	Error obtained	Rectification
1.	n+1	n-1
2.	Syntax error	Rectified

WEEK-2

A) AIM: Write a python program to perform linear search on the given input.

Program:

```
def linear_search_sorted(arr,key):
    for i in range(len(arr)):
        if arr[i]==key:
            return i
        elif arr[i]> key:
            return -1
    return -1

arr = [1,4,6,8,9,12,15,16,19]
key = int(input("enter the element to be found:"))
result=linear_search_sorted(arr,key)

if result==-1:
    print("element not found")
else:
    print(f"element found at index : {result} ")
```

Output:

```
enter the element to be found:12
element found at index : 5
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 2>
```

```
enter the element to be found:20
element not found
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complixity is O(n).
- The target element may be found at beginning or last or at middle.
- If the key is greater than every element in the sorted array, the search reaches the end and returns `-1`.

Error table:

s.no	Error obtained	Rectification
1.	Return placed in wrong place	Corrected later
2.	Calling the function	Rectified

B) Aim: Write a python program to perform binary search on the list.

Program: Unsorted

```python
#av.sc.u4cse25101
def binary_search(arr, key):
    i, j = 0, len(arr) - 1
    while i <= j:
        mid = (i + j) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            i = mid + 1
        else:
            j = mid - 1
    return -1
arr = [15, 3, 9, 20, 7, 12, 18]
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)
if result != -1:
    print(f"Element found at index {result} in the sorted array")
else:
    print(f"Element not found in the array")
```

Output:

```
Sorted array: [3, 7, 9, 12, 15, 18, 20]
Enter the element to search: 15
Element found at index 4 in the sorted array
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 2>
```

```
Sorted array: [3, 7, 9, 12, 15, 18, 20]
Enter the element to search: 25
Element not found in the array
PS C:\Users\SAI HASHINI>
```

Program: Sorted

```python
#av.sc.u4cse25101
def binary_search(arr, key):
    i, j = 0, len(arr) - 1
    while i <= j:
        mid = (i + j) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            i = mid + 1
        else:
            j = mid - 1
    return -1
arr = [3,7,9,12,15,18,20]
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)
if result != -1:
    print(f"Element found at index {result} in the
else:
    print(f"Element not found in the array")
```

Output:

```
Enter the element to search: 12
Element found at index 3 in the sorted array
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 2>
```

```
Enter the element to search: 2
Element not found in the array
```

Explanation:

- The binary search should always executed in sorted order.
- Either it can be ascending or descending order.
- So the program is in unsorted order so we need to do sorting first later the binary search is executed.

- Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A) AIM: write a python program to implement bubble sort, where the elements are taken from user.

Program:

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]


n = int(input("Enter no. of elements: "))

if n <= 0:
    print("Number of elements must be positive")
else:
    arr = []

    print("Enter elements:")
    for i in range(n):
        arr.append(int(input()))

    bubble_sort(arr)

    print("Sorted Array:")
    for element in arr:
        print(element, end=" ")
```

Output:

```
Enter no. of elements: 5
Enter elements:
12
25
46
7
97
Sorted Array:
7 12 25 46 97
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 3>
```

```
Enter no. of elements: 0
Number of elements must be positive
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>
```

Explanation:

- Bubble sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program may contain elements in **unsorted order**, so adjacent elements are compared and swapped if they are in the wrong order.
- This process is repeated for multiple passes until the **entire array is sorted**.
- For `n = 0`, the program now prints the error message without trying to use an undefined `arr`.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B) AIM: write a python program to implement insertion sort, where the elements are taken from user.

Program:

```python
#AV.SC.U4CSE25101
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key
    return arr
n = int(input("Enter no. of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted Array:")
for element in arr:
    print(element, end=" ")
```

Output:

```
Enter no. of elements: 0
Enter the elements:
Sorted Array:
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>
```

```
Enter no. of elements: 5
Enter the elements:
24
4
12
75
35
Sorted Array:
4 12 24 35 75
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 3>
```

Explanation:

- Insertion sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program takes one element at a time and **inserts it into its correct position** in the already sorted part.
- This process is repeated until **all elements are sorted**.
- When the number of elements is zero, the array is empty and insertion sort simply performs no iterations.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

C) AIM: write a python program to implement selection sort, where the elements are taken from user.

Program:

```python
#AV.SC.U4CSE25101
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]
n = int(input("Enter no. of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
print("Original Array:", arr)
selection_sort(arr)
print("Sorted Array:", arr)
```

Output:

```
Enter no. of elements: 5
Enter the elements:
24
76
35
12
9
Original Array: [24, 76, 35, 12, 9]
```

```
Enter no. of elements: 0
Enter the elements:
Original Array: []
Sorted Array: []
```

Explanation:

- Selection sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program finds the **smallest element** from the unsorted part and swaps it with the first element of that part.
- This process is repeated until the **entire array is sorted**.
- When `n = 0`, the array is empty and selection sort performs no iterations, so the program runs normally.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified